Coding-agent brief: hierarchical Bayesian lexical dating in portable C99/WebAssembly

0. Your role

Act as a senior scientific-programming engineer with expertise in:

- Bayesian computation;
- phylogenetic likelihoods;
- historical linguistics and lexical evolution;
- numerical methods in dependency-free C;
- portable WebAssembly design;
- simulation-based validation.

Build a reusable scientific library and command-line program for estimating language divergence times from basic-vocabulary cognacy data.

The system is provisionally described as **"glottochronology on steroids,"** but it must not simply modernize the classical Swadesh formula. It must expose multiple explicit probabilistic models, quantify uncertainty, learn concept-specific and family-specific rates, use historical calibrations, and report when the data cannot identify an absolute date.

The scientific core must be written in **strict C99**, depend only on the C standard library and `libm`, and compile both natively and to WebAssembly.

A separate, nonessential Python toolchain may be created for assembling reference priors, preparing datasets, validation, plotting and posterior analysis. Python must never be required by the C inference engine.

Do not attempt to infer tree topology in the first release. A topology is supplied by the user.

---

1. Non-negotiable product decisions

Implement all architectural decisions below.

1.1 Equal support for pairwise and fixed-tree dating

There are three analysis commands:

```
date-pair
date-pair-context
date-tree
```

`date-pair`

Input consists of two languages, lexical observations, reference priors and optional calibrating information.

Output is a posterior distribution for their divergence time.

`date-pair-context`

The target is still the divergence of two languages, but the analysis is performed using a larger supplied family tree and lexical dataset. The posterior for the target pair is obtained from the posterior age of its MRCA.

`date-tree`

Input consists of a fixed tree, lexical data, dated tips or node calibrations, and prior packages.

Output includes posterior distributions for all internal node ages, lexical rates, family-level rate effects and model parameters.

All three commands must use the same underlying definitions of time, lexical rates, concept priors and contamination.

1.2 Fixed topology, topology-ready architecture

Version 1 accepts one fixed rooted topology.

Internally, design the API so that a future wrapper can run the same analysis over a weighted collection of trees. Do not assume that there will always be exactly one topology in every data structure or output format.

Do not implement topology search, NNI, SPR, tree proposals or a tree prior in version 1.

1.3 Bayesian posterior distributions are essential

Maximum-likelihood estimates may be supplied for initialization and debugging, but they are not sufficient final outputs.

Every scientifically supported analysis must return posterior samples or a numerically integrated posterior.

Required posterior summaries include:

- mean;
- median;
- standard deviation;
- posterior mode where stable;
- 50%, 80%, 90% and 95% equal-tail intervals;
- 80%, 90% and 95% highest-posterior-density intervals;
- requested threshold probabilities such as `P(age > 3000)`;
- prior-to-posterior comparisons;
- diagnostics showing whether the result is prior-dominated.

1.4 External reference priors loaded dynamically

Do not hardcode concept rates or family rates in C.

The C program must load versioned, strict TSV prior packages. Reference packages will be built and improved separately, primarily with Python.

The supplied family data must be able to update the reference priors rather than merely selecting fixed constants.

1.5 Canonical concept identifiers

Use stable project-specific canonical concept IDs internally.

Initially, include mappings to Concepticon IDs when available, but do not make Concepticon a permanent architectural dependency. The project may later migrate to another ontology.

Requirements:

- canonical IDs are immutable integers or ASCII identifiers;
- aliases and external mappings live in separate tables;
- inference code uses only the internal canonical ID;
- an unknown custom concept remains valid;
- unknown concepts receive broad fallback priors;
- never match concepts silently using only approximate labels.

1.6 Strict TSV ingestion in C

The C library must contain a small dependency-free parser for the required TSV formats.

Requirements:

- input may come from a filesystem path or an in-memory buffer;
- headers are mandatory;

- fields are separated by literal tab characters;
- UTF-8 text may be stored and passed through, but the C engine need not normalize Unicode;
- `.` represents missing data;
- blank lines are ignored;
- lines beginning with `#` are comments;
- malformed numeric fields must produce useful errors;
- duplicate IDs must be detected;
- unknown required columns are errors;
- unknown optional columns may be retained or ignored with a warning;
- no quoted-field or multiline-field syntax is required;
- never call `exit()` or `abort()` inside the reusable library.

1.7 Browser-scale target

The architecture must support:

```
up to 100 languages
up to 1,000 concepts
multiple forms per language–concept cell
a rooted fixed tree
four independent posterior chains
```

The core is single-threaded. Browser applications may run chains in separate Web Workers, each with its own Wasm instance.

Design for:

- bounded memory;
- resumable/chunked MCMC execution;
- deterministic seeds;
- no recursion deep enough to overflow a normal browser stack;
- no dependence on POSIX, threads, files, locale or operating-system entropy.

---

2. Scientific scope

The system estimates dates from lexical cognacy observations under explicit evolutionary models.

It must distinguish:

1. **lexical time**, which is identifiable from lexical divergence;
2. **calendar time**, which requires calibrations or an externally anchored clock;
3. **relative concept rates**;
4. **family-wide clock variation**;

5. **branch-specific rate variation**;
6. **observation error, borrowing and incompatible lexical histories**.

If no absolute calibration or clock-scale prior is supplied, report divergence in lexical-time units. Do not fabricate years.

For example:

```
Estimated divergence: 0.83 lexical-time units
Absolute years are not identifiable from the supplied data.
Add a dated tip, node calibration, or clock-scale prior.
```

The project is not intended to prove that lexical evolution follows one universal clock. It is intended to determine what can be inferred under several transparent assumptions and how sensitive the answer is to those assumptions.

---

3. How to use the Rönchen–Wiklund–Hammarström paper

Read and document:

> Philipp Rönchen, Tilo Wiklund and Harald Hammarström.
> “Likelihood calculation in a multistate model of vocabulary evolution for linguistic dating.”
> Language Dynamics and Change 14(1), 2024, pp. 1–41.
> DOI: 10.1163/22105832-bja10032.

Do **not** reimplement their repository, reproduce its software architecture, or make their model the sole basis of this project.

Do not copy code from their implementation.

Use the paper for four purposes.

3.1 Adopt as useful ideas

Evaluate and, where appropriate, independently implement:

- meaning-wise multistate cognate representation;
- lexical replacement as transition to a novel cognate state;
- support for cognate states not observed at the leaves;
- the path-property interpretation of strict vertical inheritance;
- extendability or compatibility checking;

- efficient tree likelihood calculation;
- missing observations;
- lexical observations on internal nodes;
- nonbinary trees;
- a complexity target approximately linear in concepts and tree nodes for its strict compatible-data likelihood.

3.2 Do not inherit its limitations silently

The strict novel-state model assumes:

- one primary cognate state per meaning at each node;
- every true replacement creates a cognate class never used previously in the tree;
- no parallel lexical innovations;
- no back mutation;
- no undetected borrowing;
- no unresolved synonym coexistence;
- independent evolution among meanings;
- a tree-like inheritance process.

These must be explicit model assumptions, not project-wide truths.

3.3 Add robustness outside the paper's model

A concept incompatible with the strict novel-state model must not necessarily make the complete dataset have zero likelihood.

Implement a robust contamination component so that incompatible concepts receive:

```
L_c =
(1-\epsilon_c)L_{\text{tree},c}
+
\epsilon_c L_{\text{noise},c}.
```

Initially, allow a global contamination probability:

```
\epsilon \sim \operatorname{Beta}(a_\epsilon,b_\epsilon).
```

Later allow concept-specific contamination with hierarchical shrinkage.

The noise component must be explicit and documented. A reasonable initial model is a symmetric Dirichlet–multinomial or empirical categorical model over observed states. It is a statistical error model, not a claim that the actual event was random.

Report the posterior probability that each concept came from the contamination component.

3.4 Validate independently

For trees small enough to enumerate:

- enumerate all relevant ancestral-state configurations;
- calculate exact likelihoods independently;
- compare them with the optimized C implementation.

Do not validate only by comparing with the authors' implementation. Agreement between two implementations based on the same misunderstood recurrence is not sufficient.

---

4. Required model suite

Implement a model interface allowing several likelihood modules.

Suggested interface:

```
typedef struct gd_model_vtable {
    const char *name;
    int  (*prepare)(gd_context *, gd_error *);
    double (*log_likelihood)(gd_context *, gd_error *);
    double (*log_likelihood_concept)(
        gd_context *,
        size_t concept_index,
        gd_error *
    );
    int (*simulate)(gd_context *, gd_rng *, gd_error *);
    void (*destroy)(gd_context *);
} gd_model_vtable;
```

Do not expose internal structs unnecessarily in the public API.

4.1 Model P0: classical homogeneous baseline

Implement the classical equal-rate pairwise model only as a baseline and regression test.

For a concept with rate r and total path length d:

```
P(\text{same inherited cognate}) = e^{-rd}.
```

For two contemporaneous daughters whose divergence occurred t years ago:

```
d = 2t.
```

The historical 100-item Swadesh retention value of 0.86 per millennium corresponds to:

```
r_{\text{legacy},100}
=
-\log(0.86)
=
0.1508228897
```

replacements per lexical slot per millennium.

The historical Lees 200-item value of approximately 0.8048 retention corresponds to:

```
r_{\text{legacy},200}
=
-\log(0.8048)
=
0.2171614796
```

per millennium.

These values are **legacy benchmark presets**, not recommended universal constants.

Commands must require an explicit flag such as:

```
--model classical
--legacy-clock swadesh-100
```

Never select a legacy clock silently.

4.2 Model P1: hierarchical pairwise cognacy model

For concept c, family f and total path length d:

```
s_{f,c}(d)=\exp(-r_{f,c}d).
```

Use:

```
\log r_{f,c}
=
\mu_c+\beta_f+u_{f,c}.
```

Where:

- μ_c is the global reference log-rate for concept c;
- β_f is a family-wide log clock multiplier;
- $u_{f,c}$ is the family–concept interaction.

Suggested hierarchy:

```
\mu_c \sim N(m_c^{ref}, (s_c^{ref})^2)
```

```
\beta_f \sim N(0,\sigma_f^2)
```

```
u_{f,c}\sim N(0,\tau^2)
```

```
\sigma_f,\tau \sim \operatorname{HalfNormal}(0,s).
```

All hyperparameters must be configurable.

For tips with ages a_1 and a_2, and an MRCA age T:

```
d=(T-a_1)+(T-a_2)=2T-a_1-a_2.
```

Require:

```
T \ge \max(a_1,a_2).
```

Uncertain cognacy

Allow an input cognacy probability $y_c \in [0,1]$.

If:

```
p_c=P(\text{observed match}\mid T,r_c,\epsilon),
```

then use the soft-observation likelihood:

```
L_c=y_c p_c+(1-y_c)(1-p_c).
```

Record clearly that these probabilities are expert or upstream classification uncertainty, not learned automatically by this version.

Pairwise contamination

Allow:

```
p_c=
(1-\epsilon_c)e^{-r_cd}
+
\epsilon_c q_c,
```

where q_c is the match probability under the contamination model.

Support:

```
contamination = none
contamination = global
contamination = concept
```

4.3 Model T1: strict novel-state multistate model

This is the model most closely related to the 2024 paper, but it is only one model module.

Properties:

- one primary cognate state per language–concept observation;
- a replacement creates a new cognate class;
- a cognate class cannot originate independently twice;
- extinct cognate classes cannot reappear;
- unobserved ancestral states are permitted;
- missing observations are marginalized;
- observations may occur at leaves or labelled internal nodes;
- strict and hierarchical concept clocks are supported;
- topology is fixed;
- compatible-data likelihood must be efficient.

Implement:

1. construction of the minimal data extension or equivalent structure;
2. path-property/compatibility checks;
3. concept-wise likelihood;
4. robust contamination mixture;
5. simulation from the same model;
6. debug mode that emits the extended-state constraints.

Do not make incompatible data disappear automatically. The user may choose:

```
incompatible = error
incompatible = exclude
incompatible = contamination
```

The default is `contamination`.

4.4 Model T2: binary cognate-set model

Represent each observed cognate class as a binary trait.

This model naturally permits multiple simultaneous cognate classes for one meaning and therefore provides an alternative for synonymy and polymorphism.

Implement an asymmetric binary CTMC first:

```
0 \xrightarrow{\lambda_g} 1,
\qquad
1 \xrightarrow{\lambda_l} 0.
```

Add ascertainment correction because all-zero unobserved cognate classes are absent from the input data.

Later add a stochastic-Dollo-like birth-and-loss module, but do not delay the initial Bayesian system until the full stochastic Dollo process is available.

Important warnings:

- cognate sets belonging to one meaning are not truly independent;
- meanings with many cognate classes can contribute disproportionate likelihood;
- raw marginal likelihoods from binary and multistate encodings are not automatically comparable.

Model comparison across different encodings must use predictions transformed to a shared observable space, such as held-out pairwise cognacy or held-out language–concept states.

4.5 Optional later model: finite-state Mk/CTMC

Reserve an interface for a finite-state multistate CTMC that permits homoplasy.

Do not implement it before the strict novel-state and binary modules are validated.

Its role would be:

- robustness comparison;
- deliberate relaxation of the no-parallel-development assumption;
- examination of whether allowing homoplasy improves out-of-sample prediction.

Do not assume that greater flexibility makes it more realistic.

---

5. Synonyms and polymorphism

Input must retain all forms supplied for a language–concept cell.

Each form may have:

- a cognate class;
- a primary-word weight;
- a cognacy confidence;
- an inheritance probability;
- a loan probability;
- an annotation status.

5.1 Strict multistate analysis

The strict novel-state model has one latent primary cognate state per language–concept cell.

Support two cases.

Known primary state

One form has:

```
primary_weight = 1
```

and all others have zero weight.

Uncertain primary state

Candidate forms have weights summing to one.

In pairwise mode, marginalize exactly over candidate pairs.

In fixed-tree mode, treat the primary state as a latent categorical variable and sample it within MCMC. Update only the affected concept likelihood.

Do not perform a combinatorial enumeration over all synonym configurations.

If a cell has no primary weights, use:

```
primary_weight = 1 / number_of_candidates
```

but emit a warning.

5.2 Binary analysis

All cognate classes present in the cell are coded as present.

No form is discarded merely because it is not primary.

5.3 Genuine coexistence

The first release does not attempt a complete evolutionary model of synonym birth, competition, semantic narrowing and loss.

Document this limitation. Preserve enough input data to support such a future model.

---

6. Missing data, loans and uncertain judgments

6.1 Missing data

Missing means unknown, not absence.

A missing language–concept cell contributes no direct lexical observation.

Distinguish:

```
missing
concept absent/not lexicalized
form unknown
form intentionally excluded
```

The first release may treat all but genuine observed forms as missing in the likelihood, but their statuses must remain available in diagnostics.

6.2 Known and suspected loans

Support:

```
loan_mode = exclude
loan_mode = mixture
loan_mode = sensitivity
```

`exclude`

Forms marked as loans above a threshold are treated as missing from the vertical-inheritance likelihood.

`mixture`

Use supplied `loan_probability` to increase the observation's probability of belonging to the contamination component.

`sensitivity`

Run several analyses, for example:

```
exclude loans with p >= 0.50
exclude loans with p >= 0.80
include all with contamination
```

Do not implement explicit borrowing networks in version 1.

6.3 Alternative cognacy analyses

Allow the same dataset to contain several named cognacy codings:

```
analysis_id = expert_a
analysis_id = expert_b
analysis_id = automated_01
```

The CLI may run them separately or sample among complete alternative codings.

Do not try to infer arbitrary cognate partitions inside C in version 1.

---

7. Clocks and rate variation

Implement a progression of clock models.

7.1 Strict clock

```
r_{b,f,c}=r_{f,c}.
```

Use this as the baseline tree model.

7.2 Family clock multiplier

```
r_{f,c}=e^{\beta_f}r_c.
```

This is relevant when priors were trained across families.

7.3 Relaxed branch clock

```
r_{b,f,c}=r_{f,c}m_b
```

```
\log m_b\sim N(-\sigma_b^2/2,\sigma_b^2),
```

so that:

```
E[m_b]=1.
```

Use a shrinkage prior on σ_b.

The strict clock is the special case:

```
\sigma_b=0.
```

Branch multipliers and branch durations can be highly confounded. Require calibration sensitivity diagnostics and warn when the posterior for σ_b is weakly identified.

7.4 Optional split-pulse model

Reserve an experimental module in which a language split produces an additional lexical-change pulse:

```
H_{b,c}=r_{b,c}t_b+\delta_v.
```

Do not include it in the first validated release unless simulation demonstrates identifiability.

---

8. Calibrations and absolute time

All internal times are numeric years before a configurable origin.

Default:

```
time_origin_year = 1950
```

The C engine does not parse historical calendar strings.

Support:

```
fixed_tip
uniform_node
normal_node
lognormal_node
offset_lognormal_node
hard_minimum
hard_maximum
soft_minimum
soft_maximum
```

Each calibration must name a labelled node or tip.

Example:

```
calibration_id  target_id   type    p1  p2  p3  source_key
latin_tip   latin   fixed_tip   1900    .   .   HIST_LATIN_01
romance_root    romance_root    offset_lognormal    1400    5.30    0.25
HIST_ROMANCE_02
```

Document parameter meanings exactly.

Before inference:

- verify parent ages can exceed child ages;
- detect mutually inconsistent hard bounds;
- verify dated tips fit the tree;
- compute feasible lower and upper bounds for every node;
- fail with an informative message when no feasible chronology exists.

Required sensitivity procedures:

- prior-only sampling;
- calibration leave-one-out;
- one-calibration-at-a-time analysis;
- comparison of hard and soft versions where meaningful;
- posterior conflict reporting.

Do not silently treat a historical date as exact merely because the input gave one number.

---

9. Reference-prior system

Create a separate Python package under:

```
tools/reference_builder/
```

The resulting priors are consumed by C as TSV.

9.1 Reference modes

Support:

```
reference_mode = uninformative
reference_mode = global
reference_mode = family_specific
reference_mode = leave_current_family_out
```

Prediction mode

Default:

```
family_specific
```

when an appropriate family prior exists.

Scientific validation mode

Default:

```
leave_current_family_out
```

This avoids testing a family using priors trained partly from the same family.

Every output must record the selected mode.

9.2 Initial evidence sources

Use the following as initial sources, preserving provenance and uncertainty.

Pagel, Atkinson and Meade 2007

> “Frequency of word-use predicts rates of lexical evolution throughout Indo-European history.”
> Nature 449, 717–720.
> DOI: 10.1038/nature06176.

Use the approximately 200 meaning-specific Indo-European rate estimates as relative-rate evidence.

Do not treat them as universal absolute rates.

Map their concepts to internal canonical IDs.

Normalize relative rates by subtracting their weighted mean in log space:

```
\tilde m_c
=
\log r_c
-
\frac{\sum_c w_c\log r_c}{\sum_cw_c}.
```

Store the centered multiplier separately from the overall clock scale.

Vejdemo and Hörberg 2016

> “Semantic Factors Predict the Rate of Lexical Replacement of Content Words.”
> PLOS ONE 11(1): e0147924.
> DOI: 10.1371/journal.pone.0147924.

Use the supporting data for 167 content concepts and predictors such as:

- frequency;
- synonym count;
- number of senses;
- imageability;
- lexical class.

Use this source mainly to construct predictive priors for concepts lacking direct historical rate estimates.

Fit models with cross-validation. Do not use in-sample regression fit as evidence of prior accuracy.

Treat content and function concepts separately.

ASJP

Use a pinned version of the ASJP database as worldwide evidence about relative concept stability and broad cross-family behaviour.

Do not derive precise absolute per-year rates directly from ASJP lexical distance.

Family-balance the analysis so that large families do not dominate the worldwide prior.

Store:

- dataset version;
- retrieval date;
- checksum;
- concept mapping;
- families included;
- exclusion rules.

Rama and Wichmann 2020

> “A test of Generalized Bayesian dating: A new linguistic dating method.”
> PLOS ONE 15: e0236522.
> DOI: 10.1371/journal.pone.0236522.

Use its 30 calibration groups as a benchmark and leave-group-out validation resource.

Do not copy its fitted dating formula as the project’s clock.

9.3 Legacy overall-scale priors

Include explicit weak legacy presets:

```
prior_id    unit    log_mean    log_sd  description
legacy_swadesh100   per_kyr -1.8916490462   0.80    Weak scale prior centred on
0.86 retention/kyr
legacy_lees200  per_kyr -1.5271140564   0.80    Weak scale prior centred on
0.8048 retention/kyr
```

The wide standard deviation is intentional.

These presets are fallback anchors, not empirical conclusions.

9.4 Provisional default hyperpriors

Create configurable defaults, clearly marked provisional:

```
global clock log-SD:                0.80
minimum concept prior log-SD:       0.35
family clock SD prior:              HalfNormal(0, 0.50)
family–concept SD prior:            HalfNormal(0, 0.50)
branch-clock SD prior:              HalfNormal(0, 0.30)
global contamination prior:         Beta(1, 19)
unknown concept log-rate SD:        1.00
```

Provide sensitivity presets:

```
tight
standard
wide
```

No published result may rely on only one preset.

9.5 Published family benchmarks

Create a benchmark registry separate from inference priors.

Examples of benchmark rows may include:

- Indo-European analyses;
- the 2024 five-language Core-Indo-European case study;

- Dravidian;
- Austronesian;
- Bantu;
- Sino-Tibetan;
- Turkic;
- Uralic;
- Semitic;
- Japonic;
- selected South American families.

A benchmark row may contain:

```
family_id
study_id
root_definition
posterior_median
posterior_lower
posterior_upper
interval_probability
calibration_summary
data_availability
evidence_grade
citation
```

Do not create a numeric row unless the exact root definition and uncertainty interval have been confirmed.

Published root ages are **benchmark observations**, not automatically valid priors.

The same family may have several conflicting benchmark rows.

9.6 Evidence grades

Use:

```
A: open lexical matrix, topology, calibrations and posterior samples
B: open matrix and calibration specification, incomplete posterior output
C: published posterior summaries only
D: traditional or approximate date without reproducible analysis
```

Only A and B sources may train detailed concept-rate priors by default.

C sources are external validation.

D sources are qualitative or sensitivity references.

10. Required TSV schemas

Implement and document these files.

10.1 `languages.tsv`

```
language_id name    family_id   is_extinct  metadata
latin   Latin   indo_european   1   .
italian Italian indo_european   0   .
```

10.2 `concepts.tsv`

```
concept_id  label   external_namespace  external_id word_class  status
C000948 WATER   Concepticon 948 noun    active
C_CUSTOM_001    CUSTOM_CONCEPT  .   .   unknown active
```

10.3 `forms.tsv`

```
analysis_id language_id concept_id  form_id form    cognate_id  primary_weight
cognacy_confidence  inheritance_probability loan_probability    status
expert_a    italian C000948 f001    acqua   COG_WATER_1 1.0 1.0 0.99    0.01
observed
expert_a    english C_SMALL f002    small   COG_SMALL_1 0.75    0.95    0.98
0.02    observed
expert_a    english C_SMALL f003    little  COG_SMALL_2 0.25    0.90    0.95
0.05    observed
```

Weights apply within a language–concept cell.

Validate that nonmissing primary weights sum approximately to one.

10.4 `calibrations.tsv`

```
calibration_id  target_id   type    p1  p2  p3  is_hard source_key
latin_age   latin   fixed_tip   1900    .   .   1   HIST_001
root_prior  root    offset_lognormal    3000    7.5 0.4 0   HIST_002
```

10.5 `concept_priors.tsv`

```
concept_id  scope_id    unit    log_rate_mean   log_rate_sd n_families
n_observations  evidence_grade  source_key  version
C000948 GLOBAL  per_kyr -2.15   0.55    20  400 B   META_001    1.0.0
C000948 indo_european   per_kyr -2.32   0.32    1   87  A   PAGEL_2007  1.0.0
```

10.6 `family_priors.tsv`

```
family_id   log_clock_mean  log_clock_sd    evidence_grade  source_key  version
indo_european   0.00    0.40    B   META_IE_001 1.0.0
UNKNOWN 0.00    0.80    D   DEFAULT 1.0.0
```

10.7 `analysis.tsv`

Use one key–value pair per row:

```
key value
model   novel_state
reference_mode  family_specific
clock   hierarchical
contamination   global
chains  4
warmup  5000
samples 10000
seed    123456789
time_origin_year    1950
```

Unknown keys are errors unless explicitly namespaced:

```
x_custom_key
```

10.8 Tree input

Accept rooted Newick with labelled leaves and labelled internal nodes.

Example:

```
((italian,spanish)romance_west,romanian)romance_root;
```

Implement a minimal independent Newick parser supporting:

- labels;
- branch lengths;
- internal labels;
- polytomies;
- comments only if simple to support safely.

In dating mode, supplied branch lengths may be ignored, used as starting values, or interpreted according to an explicit configuration. Never guess.

---

11. Inference

11.1 Pairwise posterior integration

For low-dimensional pairwise models, prefer deterministic integration over MCMC.

Implement:

- adaptive grid refinement;
- log-space normalization;
- numerical CDF construction;
- adaptive Simpson or Gauss–Kronrod integration where suitable;
- Gauss–Hermite quadrature for lognormal rate integration if validated;
- exact marginalization over synonym candidates.

Return the evaluated posterior grid if requested.

Use MCMC only when pairwise mode includes enough latent variables that deterministic integration is no longer practical.

11.2 Fixed-tree MCMC

Implement a dependency-free Metropolis-within-Gibbs sampler.

Parameter blocks:

- internal node ages;
- global clock scale;
- family multiplier;
- concept rates or deviations;
- family–concept deviations;
- relaxed branch multipliers;
- contamination probability;
- latent primary synonym states;

- calibration hyperparameters when explicitly uncertain.

Suggested updates:

- log-scale random walks for positive rates;
- logit-scale updates for bounded proportions;
- age proposals constrained by parent and child ages;
- block scaling of a clade's internal ages;
- slice sampling for one-dimensional scale parameters where stable;
- categorical Gibbs or Metropolis updates for latent primary forms.

Adapt proposal scales only during warm-up.

Freeze adaptation before retained samples.

Run chains with independent deterministic seeds derived from a master seed.

11.3 Node-age parameterization

Maintain:

```
age(parent) > age(child)
```

for every edge with positive duration.

Either:

- use transformed node proportions within feasible intervals; or
- use constrained proposals that reject invalid ages.

Document the Jacobian whenever a transformed parameterization is used.

Avoid hidden hard bounds not present in the configuration.

11.4 Numerical stability

Required:

- log likelihoods throughout;
- `logsumexp`;
- `log1p`;
- `expm1`;
- stable evaluation of `log(1-exp(-x))`;
- scaling or normalization in pruning calculations;
- explicit NaN and infinity checks;

- no silent underflow to zero;
- reproducible summation order where practical.

11.5 Random-number generator

Do not use `rand()`.

Implement one documented generator such as PCG32 with:

- independent streams;
- published test vectors;
- deterministic serialization;
- uniform integers without modulo bias;
- open-interval and closed-open uniform doubles;
- normal, exponential, gamma, beta and categorical sampling.

All distributions require statistical unit tests.

11.6 Chain checkpointing

Allow:

```
gd_chain_serialize(...)
gd_chain_restore(...)
gd_chain_step(context, n_iterations, ...)
```

This is necessary for browser execution, pause/resume and responsive interfaces.

The serialization format must contain a version and checksum.

---

12. Diagnostics and outputs

12.1 MCMC diagnostics

Implement or calculate in the Python postprocessor:

- rank-normalized split-$\hat{R}$;
- bulk effective sample size;
- tail effective sample size;
- acceptance rate by proposal;
- number of invalid proposals;
- chain means and quantiles;
- autocorrelation summaries;

- Monte Carlo standard error;
- warnings for insufficient mixing.

Do not declare convergence from acceptance rate alone.

12.2 Identifiability diagnostics

Report:

- whether an absolute scale is anchored;
- posterior correlation between clock rate and root age;
- posterior correlation between branch multipliers and durations;
- concepts with posterior nearly equal to prior;
- nodes determined mainly by one calibration;
- parameters with substantial mass at configured bounds;
- prior-to-posterior information gain.

12.3 Concept diagnostics

For every concept, optionally output:

- prior rate distribution;
- posterior rate distribution;
- contamination posterior;
- compatibility with strict novel-state inheritance;
- number of observed languages;
- number of cognate classes;
- synonym count;
- loan probability summary;
- pointwise log likelihood;
- posterior predictive discrepancy.

12.4 Posterior files

Produce strict tabular outputs:

```
posterior_nodes.tsv
posterior_rates.tsv
posterior_hyperparameters.tsv
posterior_concepts.tsv
diagnostics.tsv
pointwise_loglik.tsv
run_manifest.tsv
samples.bin or samples.tsv
```

The run manifest must include:

- program version;
- Git commit when available;
- compiler;
- platform;
- Wasm/native mode;
- input checksums;
- prior-package version;
- random seed;
- configuration;
- model;
- reference mode;
- warnings;
- start and completion metadata.

12.5 Model comparison

Do not select a model based only on in-sample likelihood.

Output pointwise log likelihood by concept so that Python can calculate:

- WAIC;
- PSIS-LOO where diagnostics permit;
- held-out-concept prediction;
- held-out-language prediction;
- calibration prediction.

When data encodings differ, compare predictions in a common observable representation.

Do not compare raw AIC, BIC, WAIC or marginal likelihood across different data codings without an explicit justification.

---

13. Validation requirements

Validation is a release requirement, not a later enhancement.

13.1 Deterministic unit tests

Test:

- TSV parsing;
- Newick parsing;
- canonical mapping;
- calibration feasibility;

- RNG test vectors;
- distribution moments;
- log-space helpers;
- tree traversals;
- MRCA calculation;
- missing data;
- synonym-weight normalization;
- pairwise symmetry;
- zero path length;
- extreme old dates;
- very slow and very fast concepts.

13.2 Analytical pairwise tests

For homogeneous pairwise cases:

- compare against the classical formula;
- compare numerical integration against dense high-precision Python grids;
- test posterior normalization;
- test quantiles against analytically or numerically known results.

13.3 Brute-force tree likelihood tests

For tiny trees and small state sets:

- enumerate ancestral histories;
- calculate probability by direct summation;
- compare with pruning or recursive likelihoods;
- include missing leaves;
- include an observed internal node;
- include a polytomy;
- include compatible and incompatible patterns;
- include one and several cognate classes.

Tolerance must be documented.

13.4 Simulation-based calibration

For each supported Bayesian model:

1. draw parameters from the configured prior;
2. simulate a tree dataset;
3. fit the same model;
4. calculate posterior ranks;
5. test rank uniformity;
6. measure credible-interval coverage.

Run SBC separately for:

- pairwise dates;
- strict-clock tree dates;
- concept rates;
- family multipliers;
- relaxed clocks;
- contamination;
- latent synonym-primary states.

A model is not considered implemented merely because MCMC runs.

13.5 Parameter-recovery experiments

Create simulation grids varying:

- number of languages;
- number of concepts;
- missing-data fraction;
- number of calibrations;
- calibration precision;
- concept-rate heterogeneity;
- branch-rate heterogeneity;
- synonym frequency;
- loan/error rate;
- family depth.

Report bias, RMSE, interval width and coverage.

13.6 Posterior predictive checks

Simulate replicated datasets from posterior draws and compare:

- number of cognate classes per meaning;
- singleton and doubleton cognate classes;
- pairwise cognate-retention distribution;
- compatibility violations;
- synonym counts;
- clade-specific retention;
- concept-wise retention;
- rate variation;
- missingness-conditioned statistics;
- concentration of changes on branches.

This is essential because a model may recover its own simulated parameters while remaining inadequate for real lexical data.

13.7 Robustness injection tests

Starting from model-generated clean data, inject:

- random miscoding;
- false cognacy;
- missed cognacy;
- loans within clades;
- loans across distant branches;
- semantic-shift-like polymorphism;
- nonrandom missing data;
- erroneous calibrations.

Compare strict, contamination and binary models.

13.8 Calibration validation

Implement:

- calibration leave-one-out;
- prediction of omitted dated tips;
- prediction of omitted node constraints;
- calibration conflict diagnostics;
- sensitivity to calibration distribution shape.

13.9 Reference-prior validation

Use leave-family-out experiments.

For each held-out family:

1. build global priors without that family;
2. infer its dates and rates;
3. compare against its historical calibrations;
4. record coverage and error;
5. compare global, family-specific, legacy and weak-prior modes.

Never evaluate a family-specific prior on the same observations used to construct it without labelling the result in-sample.

---

14. C99 and WebAssembly architecture

Suggested repository structure:

```
include/
    gd.h
    gd_types.h
    gd_error.h

src/
    gd_context.c
    gd_memory.c
    gd_math.c
    gd_rng.c
    gd_tsv.c
    gd_newick.c
    gd_tree.c
    gd_data.c
    gd_calibration.c
    gd_priors.c
    gd_pairwise.c
    gd_model.c
    gd_model_novel_state.c
    gd_model_binary.c
    gd_contamination.c
    gd_quadrature.c
    gd_optimizer.c
    gd_mcmc.c
    gd_diagnostics.c
    gd_output.c
    gd_checkpoint.c

cli/
    main.c
    commands.c

wasm/
    gd_wasm.c
    demo/
        index.html
        app.js
        worker.js

tools/
    reference_builder/
    validation/
    plotting/

reference/
    raw/
    generated/
```

```
    manifests/

tests/
    unit/
    likelihood/
    simulation/
    regression/
    fixtures/

examples/
    pair_simple/
    pair_context/
    romance_tree/
    five_ie_reproduction/

docs/
    model-spec.md
    priors.md
    input-formats.md
    calibrations.md
    inference.md
    validation.md
    wasm.md
    paper-evaluation.md
```

14.1 Public API

Provide a compact opaque API:

```
typedef struct gd_context gd_context;
typedef struct gd_chain gd_chain;

gd_context *gd_context_create(const gd_options *, gd_error *);
void gd_context_destroy(gd_context *);

int gd_load_languages_tsv(
    gd_context *, const char *, size_t, gd_error *
);

int gd_load_concepts_tsv(
    gd_context *, const char *, size_t, gd_error *
);

int gd_load_forms_tsv(
    gd_context *, const char *, size_t, gd_error *
);
```

```c
int gd_load_priors_tsv(
    gd_context *, const char *, size_t, gd_error *
);

int gd_load_calibrations_tsv(
    gd_context *, const char *, size_t, gd_error *
);

int gd_load_newick(
    gd_context *, const char *, size_t, gd_error *
);

int gd_validate(gd_context *, gd_validation_report *, gd_error *);

int gd_pairwise_integrate(
    gd_context *, gd_pair_result *, gd_error *
);

gd_chain *gd_chain_create(
    gd_context *, uint64_t seed, gd_error *
);

int gd_chain_step(
    gd_chain *, uint64_t iterations, gd_progress *, gd_error *
);

int gd_chain_get_sample(
    const gd_chain *, gd_sample *, gd_error *
);
```

The Wasm wrapper may expose integer handles instead of raw pointers.

14.2 Memory management

Use:

- checked multiplication before allocation;
- explicit ownership;
- a context-level arena for analysis-lifetime memory;
- reusable scratch buffers;
- no allocation inside the innermost likelihood loop;
- no variable-length arrays dependent on user input;
- no compiler extensions.

Run native builds under sanitizers.

14.3 Compilation

Required targets:

```
make native
make test
make wasm
make wasm-demo
make benchmark
```

Compile with strict warnings:

```
-std=c99
-Wall
-Wextra
-Wpedantic
-Wconversion
-Wshadow
-Wstrict-prototypes
-Wmissing-prototypes
```

Test at least GCC, Clang and Emscripten.

---

15. Milestones

Do not attempt to implement the complete project in one undifferentiated pass.

At the end of each milestone:

- run tests;
- update documentation;
- record unresolved scientific questions;
- preserve a working build;
- produce a concise milestone report.

Milestone 0 — mathematical and software specification

Deliver:

- `docs/model-spec.md`;
- `docs/paper-evaluation.md`;
- model assumptions table;

- parameter definitions and units;
- identifiability analysis;
- final TSV schemas;
- planned public C API;
- test plan.

The paper evaluation must separate:

```
idea adopted
idea adapted
idea rejected
idea postponed
```

Acceptance:

- no ambiguous definition of rate, age or path length;
- every model has a generative description;
- absolute-time nonidentifiability is documented;
- no code copied from external implementations.

Milestone 1 — portable foundation

Implement:

- error system;
- memory helpers;
- TSV parser;
- Newick parser;
- tree structure;
- postorder/preorder traversal;
- canonical IDs;
- dataset validation;
- PCG or equivalent RNG;
- numerical helper library.

Acceptance:

- native and Wasm builds;
- parser fuzz/property tests where practical;
- sanitizer-clean native tests;
- deterministic RNG test vectors;
- no inference yet.

Milestone 2 — classical and hierarchical pairwise engine

Implement:

- classical baseline;
- concept-specific rates;
- literal two-language mode;
- missing concepts;
- uncertain cognacy;
- known tip ages;
- posterior grid/integration;
- threshold probabilities;
- pairwise simulation.

Acceptance:

- agreement with classical formula;
- posterior normalization;
- analytical/numerical regression tests;
- no MCMC required for normal low-dimensional pairwise cases.

Milestone 3 — reference-package builder

Implement Python tools to:

- ingest Pagel rate data;
- ingest Vejdemo–Hörberg supporting data;
- map concepts;
- build ASJP-derived stability summaries;
- construct broad global priors;
- emit strict TSV;
- generate provenance manifests and checksums;
- create leave-family-out packages.

Acceptance:

- fully reproducible build from pinned inputs;
- no unexplained constants;
- unknown concepts receive valid fallback priors;
- reference package passes the C validator.

Milestone 4 — hierarchical pairwise inference

Add:

- global concept priors;
- family clock effects;

- family–concept deviations;
- contamination;
- exact synonym marginalization;
- prior sensitivity presets;
- prior-dominance reporting.

Acceptance:

- SBC for pairwise dates and rates;
- recovery tests across missingness and rate heterogeneity;
- output distinguishes lexical-time and calendar-time results.

Milestone 5 — fixed-tree chronology and calibrations

Implement:

- node-age representation;
- fixed and ancient tips;
- all calibration distributions;
- feasible-age propagation;
- MRCA queries;
- chronology initialization;
- prior-only sampling.

Acceptance:

- calibration contradictions caught before MCMC;
- valid node order maintained;
- correct target-pair MRCA posterior extraction.

Milestone 6 — strict novel-state likelihood

Independently implement:

- concept-wise strict novel-state likelihood;
- path/extendability diagnostics;
- missing observations;
- internal observations;
- polytomies;
- concept simulation;
- robust contamination wrapper.

Acceptance:

- brute-force agreement on tiny trees;
- approximately linear scaling in concepts and nodes for compatible data;
- incompatible concepts receive clear diagnostics;
- no automatic wholesale concept removal.

Milestone 7 — binary polymorphic likelihood

Implement:

- binary cognate-set encoding;
- asymmetric binary CTMC;
- pruning likelihood;
- ascertainment correction;
- synonym/polymorphism support;
- shared-observable predictions.

Acceptance:

- brute-force agreement;
- simulation recovery;
- documented warning about within-meaning dependence;
- no invalid direct model comparison across incompatible codings.

Milestone 8 — fixed-tree posterior sampler

Implement:

- Metropolis-within-Gibbs;
- age proposals;
- concept-rate proposals;
- hyperparameter proposals;
- latent primary-state updates;
- multiple chains;
- warm-up adaptation;
- checkpoint/resume;
- samples and pointwise likelihood output.

Acceptance:

- SBC for a small strict-clock tree;
- reproducible chains;
- no adaptation after warm-up;
- diagnostics detect deliberately stuck chains.

Milestone 9 — relaxed clocks and robust observations

Implement:

- family multipliers;
- branch multipliers;
- hierarchical concept rates;
- global and concept contamination;

- loan modes;
- calibration leave-one-out;
- model sensitivity runner.

Acceptance:

- simulated branch-rate recovery where identifiable;
- warning when branch rates and dates remain confounded;
- robustness tests with injected loans and miscoding.

Milestone 10 — WebAssembly interface

Implement:

- Emscripten target;
- in-memory TSV/Newick loading;
- integer-handle API;
- chunked chain stepping;
- progress callbacks or polling;
- checkpoint export/import;
- Web Worker demonstration;
- browser pairwise example;
- browser fixed-tree example.

Acceptance:

- no filesystem requirement;
- no blocking requirement for an entire chain;
- identical seeded results between native and Wasm within documented floating-point tolerances;
- medium-scale memory benchmark.

Milestone 11 — adequacy and cross-family validation

Implement the complete validation suite:

- SBC;
- posterior predictive checks;
- parameter recovery;
- leave-family-out priors;
- calibration prediction;
- model comparison;
- contamination stress tests.

Use the principles from:

> Benedict King.
> “Testing the validity and adequacy of linguistic phylogenetic analyses.”

PLOS Computational Biology, 2026.
DOI: 10.1371/journal.pcbi.1014312.

Acceptance:

- a model card for every likelihood;
- clear distinction between implementation validity and empirical adequacy;
- no claim that the most flexible model is automatically best;
- documented failure regimes.

Milestone 12 — reproducible examples and release

Create:

- minimal two-language example;
- pairwise-with-context example;
- Romance fixed-tree example;
- reproduction-style five-language example inspired by, but not copied from, the 2024 paper;
- tutorial for creating a custom concept list;
- tutorial for replacing Concepticon mappings;
- native CLI manual;
- Wasm integration manual;
- scientific-methods document.

Acceptance:

- every example runs from a clean checkout;
- all inputs and outputs are versioned;
- results include uncertainty and diagnostics;
- no example presents a posterior median as a known true historical date.

---

16. Performance requirements

Measure rather than guess.

Benchmark:

```
2 languages × 100 concepts
30 languages × 200 concepts
100 languages × 1,000 concepts
```

Record separately:

- preparation time;

- one likelihood evaluation;
- one complete MCMC iteration;
- memory use;
- checkpoint size;
- native versus Wasm performance.

Performance principles:

- cache concept-local structures;
- recompute only affected concepts for concept-rate and latent-primary proposals;
- cache branch durations;
- invalidate caches explicitly;
- store tree traversal orders;
- avoid matrices sized by the theoretical infinite state space;
- keep hot data contiguous;
- use iterative tree traversal after parsing.

Do not sacrifice numerical correctness for speed before validation.

---

17. Scientific output language

Reports must avoid false precision.

Preferred:

```
Under the hierarchical strict-clock model, the posterior median
for the target divergence is 1,470 BP, with a 95% credible interval
of 820–2,510 BP. The result is moderately sensitive to the Romance
calibration and weakly sensitive to removal of the concept WATER.
```

Avoid:

```
The languages diverged in 1470 BP.
```

Every report should answer:

- which model was used;
- which calibrations set the absolute scale;
- which prior package was used;
- whether family-specific data updated the reference rates;
- how much uncertainty comes from rates versus calibrations;
- whether strict vertical inheritance fits the data;
- whether alternative models produce materially different dates;

- which concepts or calibrations exert the greatest influence.

---

18. Explicit non-goals for version 1

Do not implement:

- automatic cognate detection from raw strings;
- phonological alignment likelihoods;
- sound-change models;
- topology search;
- phylogenetic networks;
- automatic borrowing-edge inference;
- geographic diffusion;
- population-size models;
- direct CLDF parsing inside C;
- JSON parsing inside the inference core;
- HMC or NUTS;
- GPU code;
- required BLAS, GSL or LAPACK dependencies.

Preserve extension points for these features.

---

19. Required scientific references

Read and cite in the project documentation:

1. Rönchen, Wiklund & Hammarström 2024, DOI `10.1163/22105832-bja10032`.
2. King 2026, DOI `10.1371/journal.pcbi.1014312`.
3. Pagel, Atkinson & Meade 2007, DOI `10.1038/nature06176`.
4. Vejdemo & Hörberg 2016, DOI `10.1371/journal.pone.0147924`.
5. Rama & Wichmann 2020, DOI `10.1371/journal.pone.0236522`.
6. Barido-Sottani et al., “Best practices in justifying calibrations for dating language families,” DOI `10.1093/jole/lzz009`.
7. Relevant work on stochastic Dollo lexical models, missing data and lateral transfer.
8. Relevant work on Bayesian linguistic phylogenetics and relaxed clocks.
9. Documentation for the pinned ASJP release used by the reference builder.
10. Primary publications for every family benchmark included in the package.

Do not rely on summaries when the primary paper or supporting data are available.

---

20. First action

Begin with Milestone 0 only.

Before producing implementation code:

1. write the mathematical model specification;
2. give every parameter a symbol, unit, support and prior;
3. describe all likelihoods generatively;
4. state how synonyms, missing data and incompatible concepts enter each model;
5. identify nonidentifiable parameter combinations;
6. define the exact pairwise posterior;
7. define the exact fixed-tree state and MCMC target;
8. produce pseudocode for the strict novel-state likelihood;
9. produce independent brute-force validation pseudocode;
10. finalize the C public API and TSV schemas;
11. list every provisional assumption requiring later simulation;
12. present a milestone dependency graph.

Do not start broad implementation until this specification is internally consistent.